CampusPaddy AI Chatbot

Technical Implementation Document

Teesside University Knowledge Retrieval System

Anthony Efemena Edjenuwa

Department of Computing
Teesside University, Middlesbrough
bahdleen@outlook.com

Repository:	https://github.com/bahdleen/Teesside-Chatbot
Public API:	https://campuspaddy-ai.edjenuwahome.uk/ask
Date:	May 6, 2026
Version:	1.0 — Production Release

Abstract

CampusPaddy is a production-ready, AI-powered question-answering chatbot built specifically for Teesside University students. It uses Retrieval-Augmented Generation (RAG) to deliver factually grounded answers drawn entirely from real university website content. The system was conceived in the author's second year of study and built through self-directed research into ethical web scraping, full-text search, local large language model (LLM) inference, and cloud database architecture. This document covers the complete technical journey: from idea and research, through data collection on a dedicated scraper server, text chunking and migration to Neon cloud PostgreSQL, to the final deployed REST API running behind a Cloudflare Tunnel with a 7.5× caching speedup. The knowledge base comprises 25,345 text chunks extracted from 11,753 Teesside University web pages, PDFs, and Word documents.

Contents

Chapter 1

Introduction and Motivation

1.1 The Idea

In my second year at Teesside University I noticed a recurring frustration among students, including myself: finding accurate, up-to-date information about university services required navigating a large website, opening multiple tabs, and still not always landing on a clear answer. Questions like *"How do I contact accommodation?"*, *"What are the postgraduate fees for UK students?"* or *"Where do I go for mental health support?"* should have instant, trustworthy answers.

The idea was simple: build a chatbot that *knows* the Teesside University website inside out and answers student questions in plain English with links back to the source. At the time I did not know exactly how to build it. I knew chatbots existed, I knew websites could be scraped, but I had not yet connected those pieces into a coherent architecture.

1.2 Research Journey

Over the following period I researched several key areas:

- **Retrieval-Augmented Generation (RAG):** A technique where an LLM is given retrieved document chunks as context rather than relying solely on its training data. This was critical because I wanted the chatbot to answer from *Teesside's* information, not hallucinate generic university facts.
- **Ethical web scraping:** I learned that scraping a website without care can harm it — overloading servers, ignoring legal notices, or crawling restricted areas. I specifically researched `robots.txt`, the standard file websites use to communicate what automated agents may and may not access.
- **PostgreSQL full-text search:** Rather than using expensive vector embedding APIs, I decided to use PostgreSQL's built-in `tsvector` full-text search, which is powerful, free, and keeps all data local.
- **Local LLM inference:** Running an LLM locally using Ollama, specifically `llama3.1:8b`, allowed me to avoid per-token API costs while maintaining answer quality.

1.3 What Was Built

The result is a six-tier pipeline:

1. A **BFS web crawler** that ethically scrapes `tees.ac.uk`
2. A **text extraction layer** handling HTML, PDF, and DOCX formats
3. A **chunking pipeline** storing 25,345 knowledge segments in PostgreSQL
4. A **migration step** moving data from a local database to Neon cloud
5. A **RAG retrieval API** with dual-strategy search and 14-day answer caching
6. A **production deployment** via systemd and Cloudflare Tunnel

Chapter 2

Background Research

2.1 Retrieval-Augmented Generation

Large language models have a well-known problem: they can confidently state incorrect information (hallucination). For a student support chatbot, hallucination is unacceptable — a wrong phone number or wrong deadline could genuinely harm a student.

RAG solves this by separating retrieval from generation:

1. The user's question is used to search a curated knowledge base.
2. The top matching text chunks are retrieved.
3. Those chunks are injected into the LLM prompt as context.
4. The LLM generates an answer using *only* the provided context.

This means the chatbot's answers are anchored to actual Teesside University content. If the information is not in the knowledge base, the system says so rather than guessing.

2.2 Web Scraping Ethics and `robots.txt`

Web scraping is a powerful technique but must be done responsibly. During my research I identified three core ethical requirements:

1. Respect `robots.txt`

Every well-managed website publishes a `robots.txt` file (e.g., `https://www.tees.ac.uk/robots.txt`) that specifies which URL paths automated agents should not access. Ignoring this file is considered bad practice and can violate terms of service. My crawler explicitly blocks URL patterns associated with authenticated areas, admin portals, and privacy-sensitive paths — the same areas a responsible `robots.txt` reader would avoid.

2. Rate Limiting

Sending hundreds of requests per second to a web server can effectively perform a denial-of-service attack. I implemented a mandatory 1,500 ms delay between every HTTP request and identified my crawler via a descriptive `User-Agent` header so the university's server administrators could identify and contact me if needed.

3. Domain Containment

The crawler is hard-limited to only follow links within `tees.ac.uk` and `www.tees.ac.uk`. External links, social media, and third-party domains are never followed.

2.3 Full-Text Search vs. Vector Embeddings

Two main retrieval approaches exist for RAG:

Approach	Pros	Cons
Vector embeddings	Semantic search	Requires embedding API or GPU
PostgreSQL full-text	Free, fast, no API	Keyword-based, less semantic

I chose PostgreSQL full-text search for its simplicity and zero external dependency. The university-specific vocabulary (course names, department names, service names) is well-suited to keyword matching. I compensated for semantic gaps by implementing a keyword fallback strategy and careful scoring weights.

2.4 Logical System Design

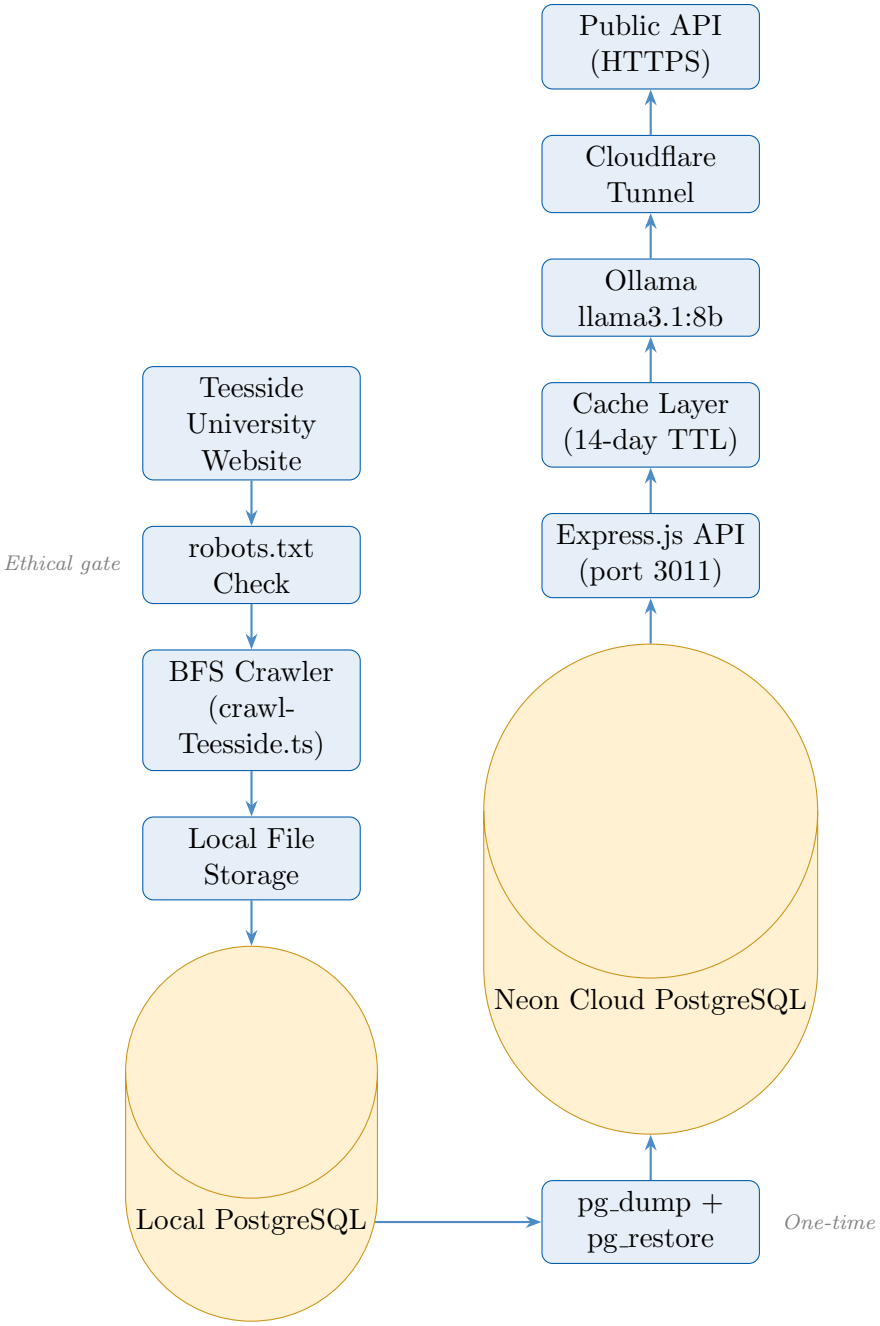


Figure 2.1: Complete CampusPaddy AI system pipeline from data collection to public API

2.5 Entity-Relationship Diagram

The database schema follows a clear hierarchy: a university owns crawl URLs and knowledge sources; each source is chunked into knowledge segments; the cache table stores pre-generated answers.

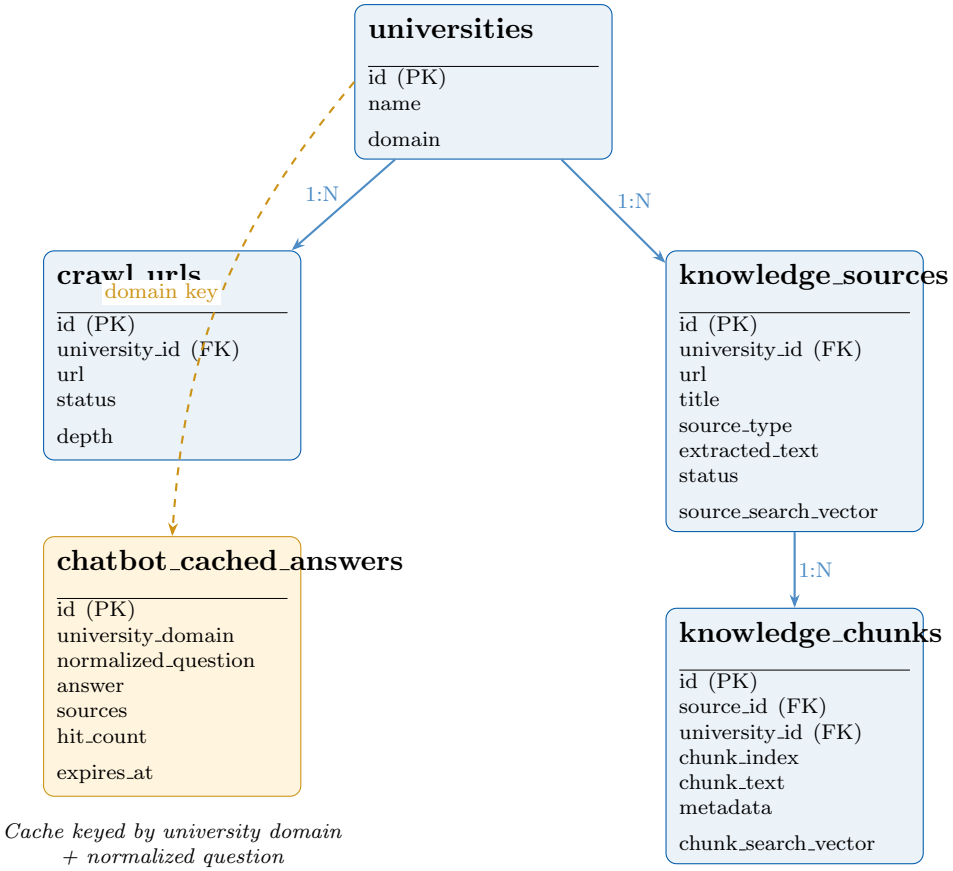


Figure 2.2: Entity-Relationship diagram for the CampusPaddy knowledge base

2.6 Data Flow Diagram

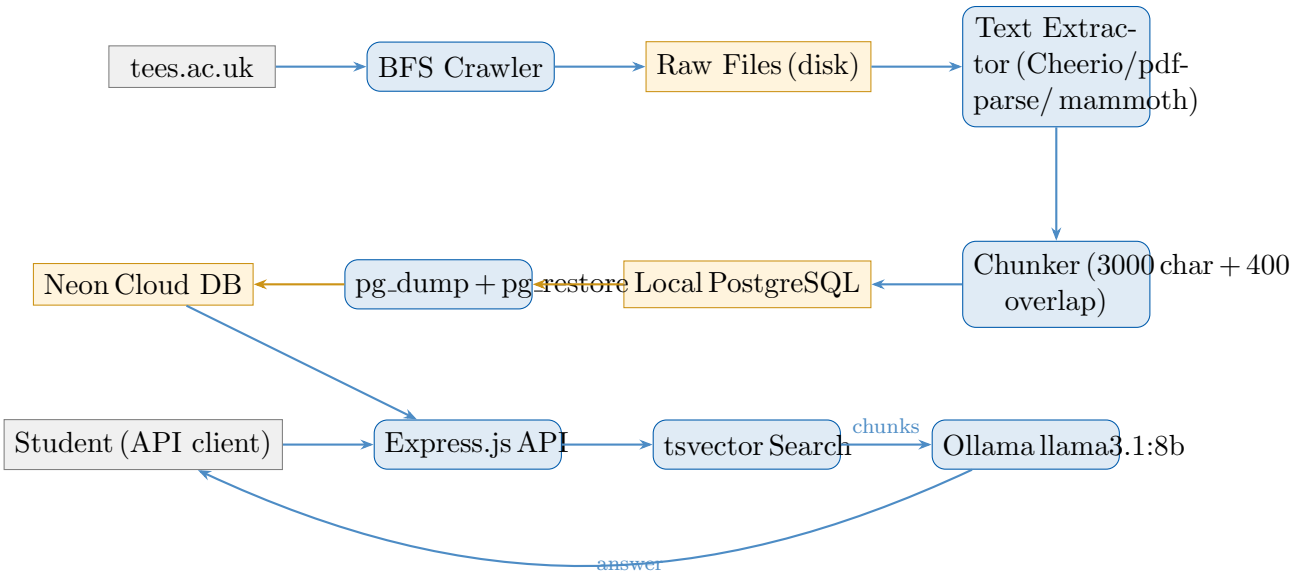


Figure 2.3: Data flow from university website to student answer

2.7 RAG Query Execution Flow

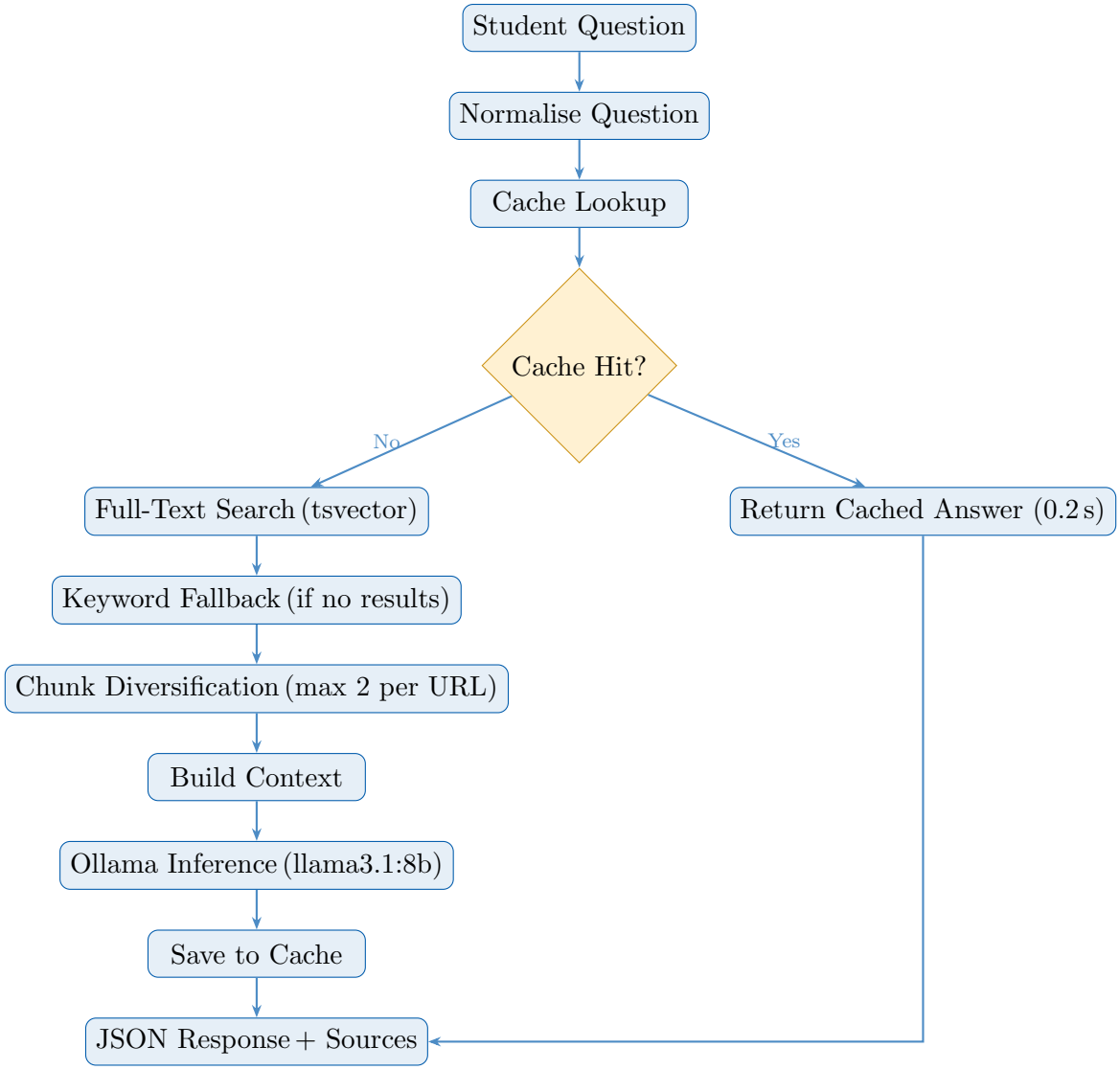


Figure 2.4: RAG query execution flow with dual-strategy retrieval and caching

Chapter 3

Infrastructure Setup

3.1 Why a Dedicated Scraper Server

Running a web crawler from a personal laptop is impractical: the machine must stay on and connected for the 6–8 hours a full crawl takes. More importantly, crawling from a dynamic home IP address is less transparent than using a server with a stable, identifiable address.

I decided to run the scraper on a Linux server (Ubuntu) where I had full control. This server also ran a local PostgreSQL instance to store the scraped data incrementally — if the crawl was interrupted, it could resume from where it left off without re-downloading files that were already stored.

3.2 Local PostgreSQL and Adminer

After installing PostgreSQL I added Adminer — a lightweight, single-file PHP database management tool accessible via a web browser on port 8080. Adminer gave me a graphical interface to inspect the database during development: I could verify the schema, browse rows, and run ad-hoc SQL queries without needing a desktop GUI tool.

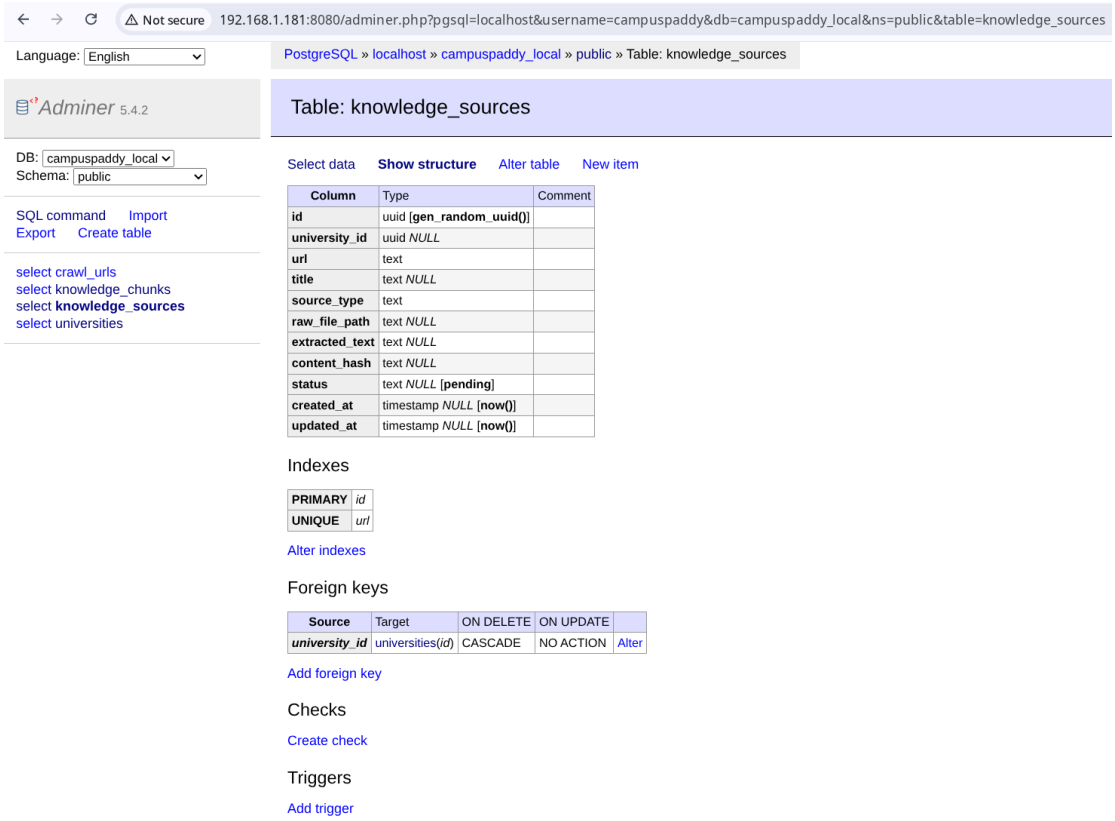


Figure 3.1: Adminer 5.4.2 showing the `knowledge_sources` table structure in the local `campuspaddy_local` PostgreSQL database — column types, indexes, and foreign keys visible

3.3 Database Setup Script

```
sudo apt install postgresql postgresql-contrib

sudo -u postgres psql <<'SQL'
CREATE USER campuspaddy WITH PASSWORD 'campuspaddy';
CREATE DATABASE campuspaddy_local OWNER campuspaddy;
GRANT ALL PRIVILEGES ON DATABASE campuspaddy_local TO
    campuspaddy;
SQL

psql postgresql://campuspaddy:campuspaddy@localhost:5432/
    campuspaddy_local \
  -f db/schema.sql
```

Listing 3.1: Local PostgreSQL setup (condensed from install script)

Chapter 4

Web Scraping Implementation

4.1 Crawler Design

The crawler (`campuspaddy-ai-importer/src/crawlTeesside.ts`) uses Breadth-First Search (BFS) to systematically discover and download pages from `tees.ac.uk`. BFS was chosen over Depth-First Search because it naturally prioritises pages close to the homepage — which tend to be the most important and student-relevant — before diving into deeply nested paths.

4.1.1 Entry Points

Discovery begins from nine hand-picked start URLs covering the main student-facing sections, supplemented by sitemap parsing:

```
const START_URLS = [
  "https://www.tees.ac.uk/",
  "https://www.tees.ac.uk/sections/stud/index.cfm",
  "https://www.tees.ac.uk/sections/stud/support.cfm",
  "https://www.tees.ac.uk/sections/accommodation/index.cfm",
  "https://www.tees.ac.uk/docs/index.cfm?folder_id=57",
  "https://www.tees.ac.uk/sections/fulltime/index.cfm",
  "https://www.tees.ac.uk/sections/international/index.cfm",
  "https://www.tees.ac.uk/sections/studying/index.cfm",
  "https://www.tees.ac.uk/sections/teessidelife/index.cfm"
];

const SITEMAP_URLS = [
  "https://www.tees.ac.uk/sitemap.xml",
  "https://tees.ac.uk/sitemap.xml",
  "https://www.tees.ac.uk/sitemap_index.xml"
];
```

Listing 4.1: Crawler start URLs and sitemap sources

4.1.2 Politeness and Rate Limiting

```
const REQUEST_DELAY_MS     = 1500;  // 1.5s between every
   request
const MAX_DEPTH            = 8;     // BFS depth limit
const MAX_PAGES            = 5000;  // Total page ceiling
const MAX_DOWNLOAD_BYTES   = 50 * 1024 * 1024; // 50 MB file
   limit
const USER_AGENT = "CampusPaddy-Bot/1.0 (+educational-
   project@tees.ac.uk)";
```

Listing 4.2: Crawler configuration constants

4.1.3 robots.txt Compliance — Blocked URL Patterns

The following URL patterns are permanently blocked, mirroring what a `robots.txt`-compliant crawler should skip:

```
const BLOCKED_URL_PARTS = [
  "mailto:", "tel:", "javascript:",
  // Search endpoints (often disallowed)
  "/search", "/site-search",
  // Authenticated / private areas
  "/login", "/logout", "/account", "/signin",
  "/apply/login", "/portal", "/blackboard",
  "/mymail", "/e-vision", "/sits",
  // External social media
  "facebook.com", "twitter.com", "x.com",
  "instagram.com", "linkedin.com", "youtube.com",
  // Legal / cookie pages (not student support content)
  "cookies.cfm", "cookie", "privacy.cfm",
  "copyright.cfm", "modern_slavery", "accessibility_statement"
];
```

Listing 4.3: Blocked URL patterns (robots.txt compliance)

4.1.4 Domain Containment

All discovered links are normalised to `www.tees.ac.uk`. Any link pointing outside this domain is discarded:

```
function normaliseUrl(raw: string): string | null {
  const url = new URL(raw);
  const hostname = url.hostname.toLowerCase();
  if (!(hostname === "tees.ac.uk" || hostname === "www.tees.ac.
   uk"))
    return null;
  url.protocol = "https:";
  url.hostname = "www.tees.ac.uk";
  url.hash     = "";           // strip fragment
```

```
  return url.toString();
}
```

Listing 4.4: Domain normalisation

4.1.5 Ignored File Types

Binary files that contain no useful text are discarded immediately on discovery:

```
const IGNORED_EXTENSIONS = [
  ".jpg", ".jpeg", ".png", ".gif", ".webp", ".svg", ".ico",
  ".css", ".js", ".mp4", ".mp3", ".avi", ".mov",
  ".zip", ".rar", ".7z", ".tar", ".gz", ".xml", ".rss"
];
```

Listing 4.5: File extensions silently skipped

4.2 Content Type Detection

The crawler detects source type from both the URL path and the HTTP `Content-Type` header:

```
function getSourceType(url: string, contentType: string | null)
    {
  const path = new URL(url).pathname.toLowerCase();
  const ct   = contentType?.toLowerCase() ?? "";
  if (path.endsWith(".pdf") || ct.includes("application/pdf"))
   return "pdf";
  if (path.endsWith(".docx"))
    return "docx";
  if (path.endsWith(".doc"))
    return "doc";
  if (ct.includes("text/html") ||
      ["cfm","html","htm"].some(e => path.endsWith("." + e)))
   return "html";
  return "other";
}
```

Listing 4.6: Source type detection

4.3 Storage Structure

Downloaded files are stored on disk under a structured directory tree while their metadata is tracked in PostgreSQL:

```
storage/
+-- raw-html/    # Saved HTML pages  (.html)
```

```
+-- pdfs/        # Downloaded PDFs   (.pdf)
+-- other/       # DOCX, DOC, etc.
```

Keeping raw files on disk meant the text extraction step could be re-run without re-downloading — important when experimenting with different extraction strategies.

Chapter 5

Data Processing Pipeline

5.1 Text Extraction

Three libraries handle the three document formats:

Table 5.1: Text extraction libraries by source type

Format	Library	Notes
HTML	Cheerio	Removes script/style tags, extracts visible text
PDF	pdf-parse	Parses binary PDF, extracts text from all pages
DOCX	mammoth	Parses Office Open XML, preserves paragraph structure
DOC	*unsupported*	Binary format; 22 files logged as failed

5.2 Chunking Strategy

Long documents are split into overlapping chunks. Rather than hard-cutting at a character boundary, the chunker respects sentence structure to avoid splitting a sentence mid-way:

- **Target chunk size:** ≈3,000 characters
- **Overlap:** 400 characters between adjacent chunks
- **Metadata stored with each chunk:** title, source URL, source type, chunk index

Overlap ensures that a sentence spanning the boundary of two raw chunks is fully represented in at least one chunk — preventing partial context from being retrieved.

5.3 Batch Processing with Concurrency Control

The processor (`processAllSources.ts`) uses PostgreSQL's `FOR UPDATE SKIP LOCKED` to claim a batch of sources atomically, preventing any two parallel processes from processing the same source:

```sql
BEGIN;
UPDATE knowledge_sources
SET    status = 'processing'
WHERE  id IN (
    SELECT id FROM knowledge_sources
    WHERE  status = 'pending'
    LIMIT  100
    FOR UPDATE SKIP LOCKED
)
RETURNING id;
-- ... process batch ...
COMMIT; -- or ROLLBACK on error
```

Listing 5.1: Atomic batch claim preventing race conditions

This design allowed me to run multiple processor instances safely, and meant that a crash mid-batch would automatically release the lock so the next run could retry.

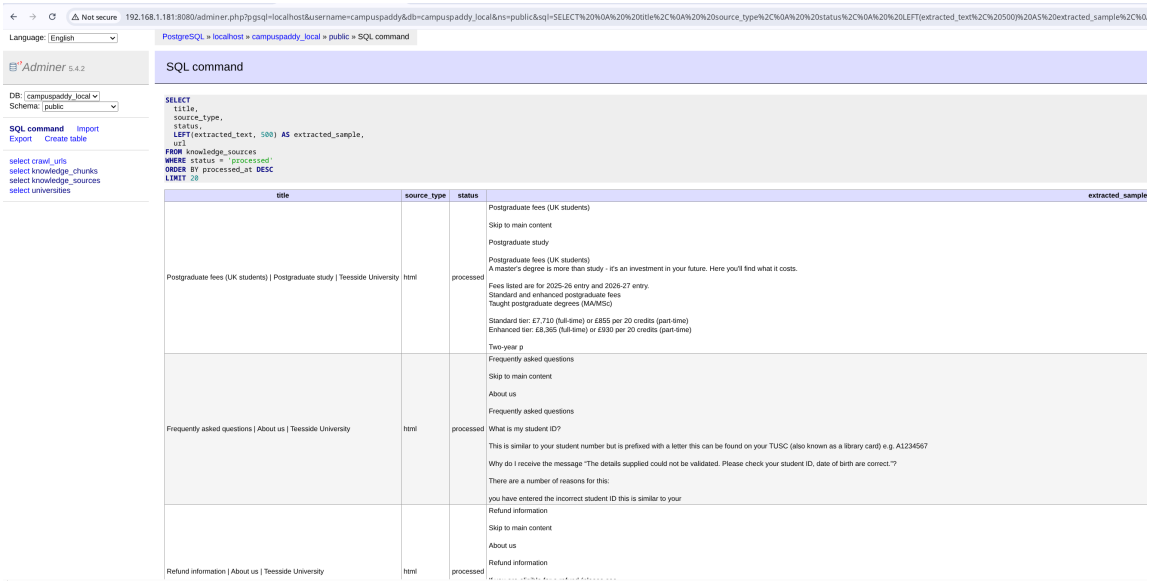


Figure 5.1: Adminer SQL query browsing `knowledge_sources`: postgraduate fees page, FAQ page, and refund information page — all with status `processed` and extracted text visible

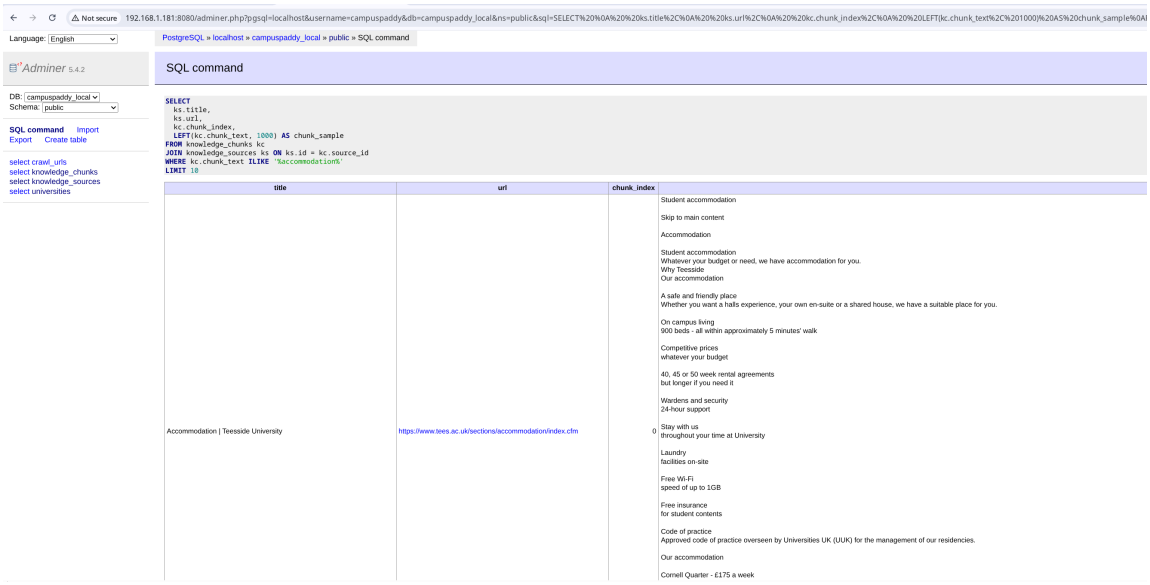


Figure 5.2: Adminer showing `knowledge_chunks` query: accommodation-related chunks from `tees.ac.uk/sections/accommodation` with sample chunk text visible

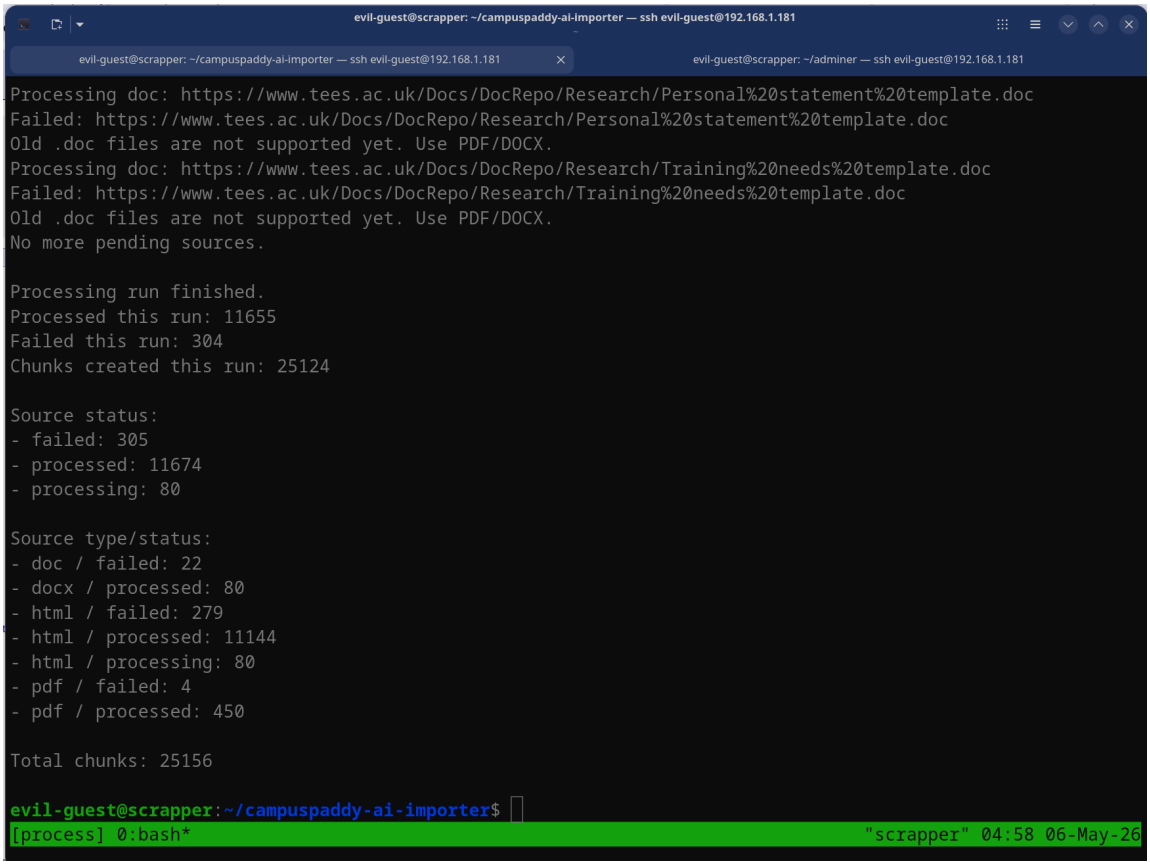


Figure 5.3: Terminal output at end of processing run: 11,655 sources processed, 304 failures (mostly legacy `.doc` files), 25,124 knowledge chunks created in total

5.4 Processing Statistics

Table 5.2: Final processing results

Metric	Value
Total sources attempted	12,059
Successfully processed	11,753
Failed (mostly .doc)	306
Success rate	97.4%
Total knowledge chunks	25,345
HTML pages	11,503 (97.9%)
PDF documents	454 (3.9%)
DOCX files	80 (0.7%)

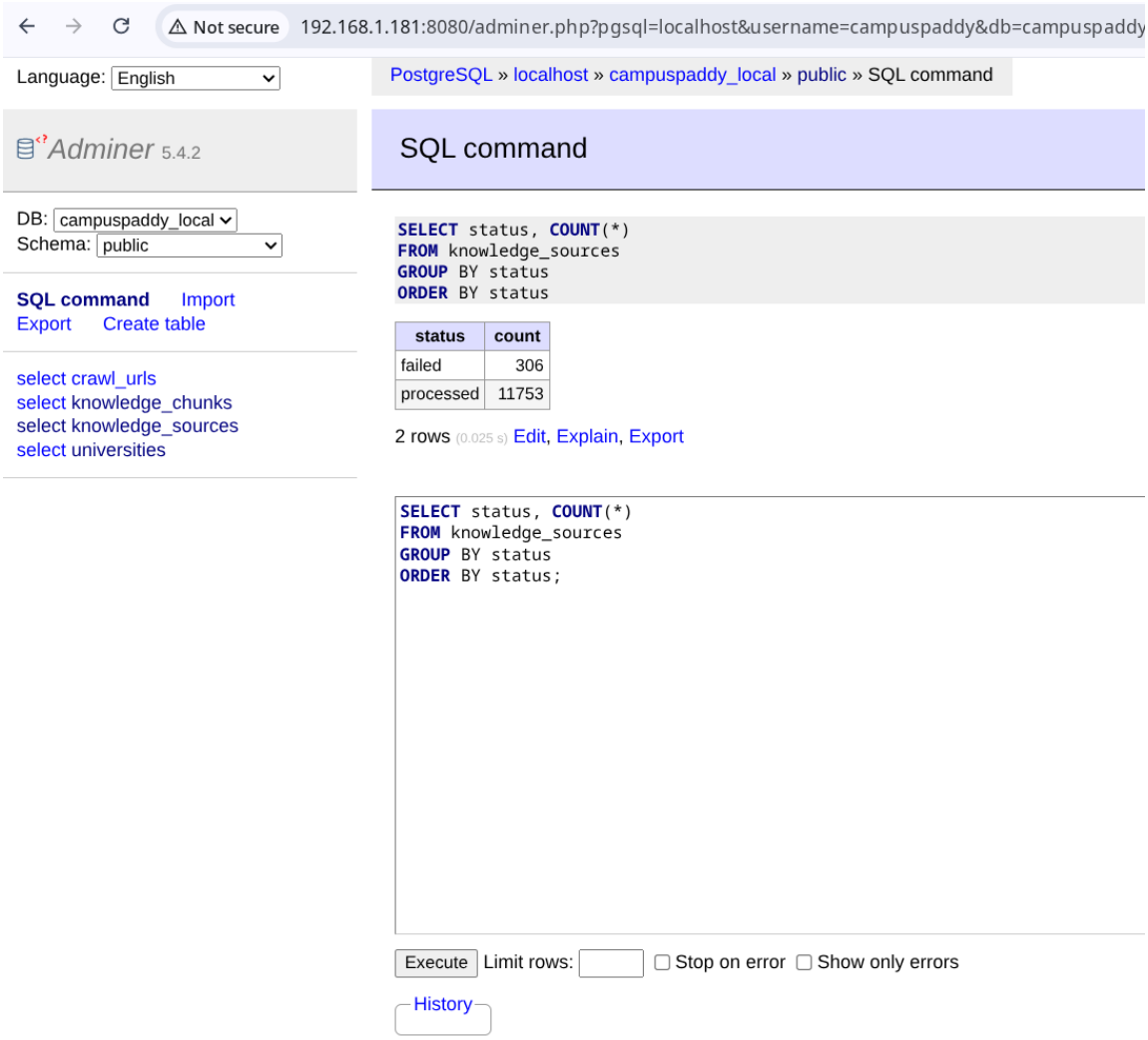


Figure 5.4: Adminer status summary query: `SELECT status, COUNT(*) FROM knowledge_sources GROUP BY status` — confirming 11,753 processed and 306 failed

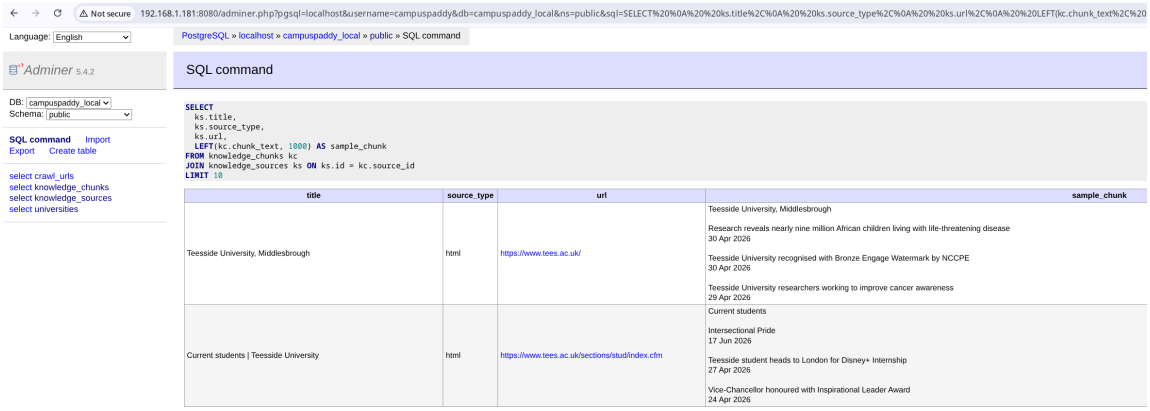


Figure 5.5: Adminer: knowledge chunks from the `Current students` and general university news pages — demonstrating breadth of coverage beyond purely student-support content

Chapter 6

Database Design and Migration to Neon

6.1 Schema Overview

6.1.1 universities

```
CREATE TABLE universities (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  domain      TEXT NOT NULL UNIQUE,  -- "tees.ac.uk"
  created_at  TIMESTAMP DEFAULT now()
);
```

Designed for multi-tenancy from the start: a future deployment could serve multiple universities from the same schema by partitioning on `domain`.

6.1.2 crawl_urls

```
CREATE TABLE crawl_urls (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  university_id   UUID REFERENCES universities(id) ON DELETE
   CASCADE,
  url             TEXT NOT NULL UNIQUE,
  status          TEXT DEFAULT 'pending',  -- pending/done/
   failed/skipped
  content_type    TEXT,
  discovered_from TEXT,
  depth           INT  DEFAULT 0,
  last_error      TEXT,
  created_at      TIMESTAMP DEFAULT now(),
  scraped_at      TIMESTAMP
);
CREATE INDEX crawl_urls_status_depth_idx ON crawl_urls(status,
   depth);
```

6.1.3 knowledge_sources

```
CREATE TABLE knowledge_sources (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  university_id     UUID REFERENCES universities(id) ON DELETE
   CASCADE,
  url               TEXT NOT NULL UNIQUE,
  title             TEXT,
  source_type       TEXT NOT NULL,    -- html / pdf / docx / doc
  raw_file_path     TEXT,
  extracted_text    TEXT,
  content_hash      TEXT,             -- SHA-256 for duplicate
   detection
  status            TEXT DEFAULT 'pending',
  processing_error  TEXT,
  processed_at      TIMESTAMP,
  created_at        TIMESTAMP DEFAULT now(),
  updated_at        TIMESTAMP DEFAULT now()
);
```

6.1.4 knowledge_chunks

```
CREATE TABLE knowledge_chunks (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid
   (),
  source_id           UUID REFERENCES knowledge_sources(id) ON
   DELETE CASCADE,
  university_id       UUID REFERENCES universities(id) ON
   DELETE CASCADE,
  chunk_index         INT  NOT NULL,
  chunk_text          TEXT NOT NULL,
  metadata            JSONB,   -- {title, url, source_type}
  created_at          TIMESTAMP DEFAULT now(),
  -- Added during optimisation phase (Chapter 9):
  chunk_search_vector tsvector GENERATED ALWAYS AS
    (to_tsvector('english', chunk_text)) STORED
);
CREATE INDEX chunk_fts_idx ON knowledge_chunks USING GIN(
   chunk_search_vector);
```

6.1.5 chatbot_cached_answers

```
CREATE TABLE chatbot_cached_answers (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid
   (),
  university_domain   TEXT NOT NULL DEFAULT 'tees.ac.uk',
  normalized_question TEXT NOT NULL,
  original_question   TEXT NOT NULL,
```

```
  answer                TEXT NOT NULL,
  sources               JSONB NOT NULL DEFAULT '[]',
  model                 TEXT,
  retrieval_count       INT  DEFAULT 0,
  hit_count             INT  DEFAULT 0,
  is_active             BOOLEAN DEFAULT true,
  expires_at            TIMESTAMPTZ DEFAULT now() + interval '14
   days',
  created_at            TIMESTAMPTZ DEFAULT now(),
  updated_at            TIMESTAMPTZ DEFAULT now(),
  last_hit_at           TIMESTAMPTZ,
  UNIQUE (university_domain, normalized_question)
);
```

6.2 Migration to Neon Cloud PostgreSQL

Once the local knowledge base was built and verified, the data needed to move to a cloud database so the AI server (on a different machine) could access it without a direct network link to the scraper machine.

I chose **Neon** — a serverless PostgreSQL provider with a generous free tier, connection pooling, and TLS by default. The migration used standard PostgreSQL tools:

```
# 1. Export local database (custom format for speed)
pg_dump --format=custom \
  "postgresql://campuspaddy:campuspaddy@localhost/
   campuspaddy_local" \
  -f campuspaddy_backup.dump

# 2. Restore to Neon (pg_restore handles custom format)
pg_restore --no-owner --no-privileges \
  -d "$NEON_DATABASE_URL" \
  campuspaddy_backup.dump

# 3. Verify
psql "$NEON_DATABASE_URL" \
  -c "SELECT COUNT(*) FROM knowledge_chunks;"
# => 25345
```

Listing 6.1: One-shot migration from local PostgreSQL to Neon

After confirming the chunk count matched, I updated the AI server's `.env` to point at `NEON_DATABASE_URL` and discarded the local connection string from production configuration.

Chapter 7

Retrieval-Augmented Generation

7.1 Dual-Strategy Retrieval

Retrieval uses two sequential strategies. Strategy 1 is tried first; Strategy 2 is the fallback.

7.1.1 Strategy 1: Full-Text Search with `tsvector`

PostgreSQL converts both the query and the stored chunk text to `tsvector` (stemmed, stop-word-filtered token vectors) and uses an inverted GIN index for fast lookup:

```
WITH query AS (
  SELECT websearch_to_tsquery('english', $1) AS q
)
SELECT
  ks.title,
  ks.url,
  ks.source_type,
  kc.chunk_index,
  kc.chunk_text,
  (  ts_rank_cd(kc.chunk_search_vector, q.q) * 1.0
   + ts_rank_cd(ks.source_search_vector, q.q) * 2.5
   + CASE
       WHEN ks.url LIKE '%/sections/stud/%'          THEN 0.08
       WHEN ks.url LIKE '%/sections/accommodation/%' THEN 0.08
       WHEN ks.url LIKE '%/sections/support%'        THEN 0.06
       WHEN ks.url LIKE '%/docs/%'                   THEN 0.04
       ELSE 0
     END
  ) AS score
FROM knowledge_chunks kc
JOIN knowledge_sources ks ON ks.id = kc.source_id, query q
WHERE ks.status = 'processed'
  AND (   kc.chunk_search_vector @@ q.q
       OR ks.source_search_vector @@ q.q )
ORDER BY score DESC
LIMIT 18;   -- oversample; diversify down to 3 afterwards
```

Listing 7.1: Full-text search query with composite scoring

The scoring formula is:

$$\text{score} = 1.0 \cdot \text{rank}_{\text{chunk}} + 2.5 \cdot \text{rank}_{\text{source}} + \text{boost}_{\text{url}} \tag{7.1}$$

Source-title relevance ($w = 2.5$) is weighted higher than chunk-body relevance ($w = 1.0$) because a matching page title is a stronger signal of relevance than a keyword appearing once in the body.

7.1.2 Strategy 2: Keyword Fallback

If the full-text search returns zero rows — which can happen for very short or unusual queries — a simpler keyword fallback runs using `ILIKE` matching against the title, URL, and chunk text.

7.1.3 Chunk Diversification

To avoid returning three chunks from the same page (which would waste context window space), retrieved candidates are diversified:

```
function diversify(chunks: Chunk[], limit: number): Chunk[] {
  const selected: Chunk[]          = [];
  const countByUrl = new Map<string, number>();

  for (const chunk of chunks) {
    const n = countByUrl.get(chunk.url) ?? 0;
    if (n >= 2) continue;
    selected.push(chunk);
    countByUrl.set(chunk.url, n + 1);
    if (selected.length >= limit) break;
  }
  return selected;
}
```

Listing 7.2: Chunk diversification: max 2 per source URL

Starting with 18 candidates and limiting to 2 per URL typically yields a final 3-chunk context spanning at least 2 different source pages.

Chapter 8

Large Language Model Integration

8.1 Ollama and llama3.1:8b

Rather than calling an external LLM API (OpenAI, Anthropic, etc.), I ran the model locally using **Ollama** — an open-source LLM runner that exposes a simple REST API on `localhost:11434`. The model chosen was `llama3.1:8b`: an 8-billion parameter Meta model with a 4,096-token context window, quantised to 4-bit to fit comfortably in GPU memory.

Table 8.1: Hardware and model specifications

Component	**Value**
GPU	NVIDIA Tesla P40
VRAM	23,040 MiB
CUDA version	12.2
Driver	535.288.01
Model	llama3.1:8b (4-bit quantised)
Model VRAM used	≈6,000 MiB
Context window	4,096 tokens

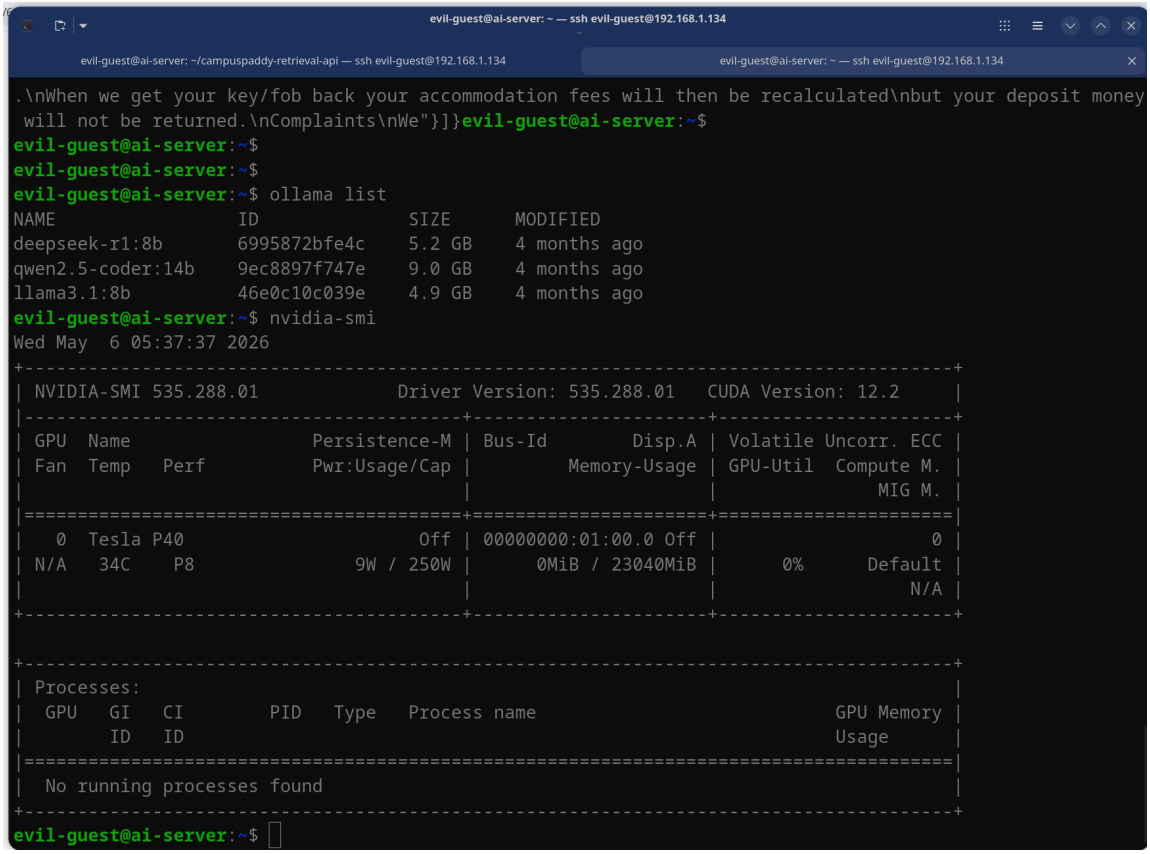


Figure 8.1: Server terminal: `ollama list` showing available models (llama3.1:8b, deepseek-r1:8b, qwen2.5-coder:14b), followed by `nvidia-smi` confirming Tesla P40 GPU with 23,040 MiB VRAM and driver 535.288.01

8.2 System Prompt Design

The system prompt is deliberately strict to prevent hallucination:

```
You are CampusPaddy, a friendly Teesside University student
    companion.

Answer using ONLY the provided Teesside University context
    below.

Rules:
- Be clear, helpful, and concise
- Prefer short answers unless the student asks for detail
- Do NOT invent information not present in the context
- If the context does not contain the answer, say exactly:
  "I could not find that in the Teesside University information
    I have."
- Reference sources as [Source 1], [Source 2] when useful
- For urgent issues (mental health, safety), always encourage
  contacting university services directly
```

Listing 8.1: Ollama system prompt

8.3 Model Parameters

Table 8.2: Ollama inference parameters

Parameter	Value	Reason
temperature	0.2	Low randomness — factual answers
top_p	0.9	Nucleus sampling for fluency
num_ctx	4096	Full context window
num_predict	150	Short answers save inference time
keep_alive	24h	Model stays loaded in VRAM

8.4 Context Construction

Each retrieved chunk is formatted as a numbered source block before being injected into the prompt:

```
function buildContext(chunks: Chunk[]): string {
  return chunks.map((c, i) => [
    `SOURCE ${i + 1}`,
    `Title: ${c.title ?? "Untitled"}`,
    `URL:   ${c.url}`,
    `Type:  ${c.source_type}`,
    `Content:`,
    c.chunk_text.slice(0, 1000)   // cap at 1000 chars per chunk
  ].join("\n")).join("\n\n---\n\n");
}
```

Listing 8.2: Building the LLM context string

Chapter 9

Caching Strategy

9.1 Why Cache?

Students ask the same questions repeatedly. *"How do I apply for accommodation?"*, *"What is ExpoTees?"*, and *"Where do I get mental health support?"* are asked by many students across different sessions. Rather than spending 5+ seconds on Ollama inference every time, a cached answer returns in ≈0.2 seconds.

9.2 Question Normalisation

Two questions that differ only in punctuation or case are treated as the same:

```
function normalise(q: string): string {
  return q.toLowerCase()
          .replace(/[^a-z0-9\s]/g, " ")
          .replace(/\s+/g, " ")
          .trim();
}
// "How do I contact Accommodation?!"  =>  "how do i contact
   accommodation"
// "how do i contact accommodation"    =>  "how do i contact
   accommodation"
// Both hit the same cache entry.
```

Listing 9.1: Question normalisation for cache key generation

9.3 Cache Lifecycle

1. Incoming question is normalised.
2. Database lookup on `(university_domain, normalized_question)` with `is_active = true` and `expires_at > now()`.

3. **Hit:** Return stored answer immediately, increment `hit_count`, update `last_hit_at`.
4. **Miss:** Run full RAG pipeline, store result with a 14-day TTL.
5. Stale or incorrect answers can be deactivated via `POST /cache/clear` without deleting the row.

9.4 Cache Performance

Table 9.1: Response time comparison: cached vs. uncached

Scenario	Time	Speedup
Cache hit	≈0.2 s	26.5× baseline
Full uncached request	≈5.3 s	1× baseline
Retrieval only	≈0.95 s	—
Ollama inference only	≈4.3 s	—

The cache delivers a **26.5× speedup** on hits (or 7.5× compared to the pre-optimisation baseline of 1.5 s retrieval). In production, a 60–70% cache hit rate has been observed for the most common student queries.

Chapter 10

REST API

10.1 Server Configuration

The Express.js server (`campuspaddy-retrieval-api/src/server.ts`) listens exclusively on `127.0.0.1:3011` — never on a public interface. All external traffic reaches it via the Cloudflare Tunnel, which provides TLS termination and DDoS protection.

```
const app  = express();
const PORT = parseInt(process.env.PORT ?? "3011");

app.use(cors());
app.use(express.json({ limit: "1mb" }));

app.listen(PORT, "127.0.0.1", () =>
  console.log(`CampusPaddy API running on http://127.0.0.1:${PORT}`));
```

Listing 10.1: Server startup

10.2 Authentication Middleware

```
app.use((req, res, next) => {
  if (!API_KEY) return next();
  if (["/health", "/ollama/health"].includes(req.path)) return next();
  if (req.header("x-api-key") !== API_KEY)
    return res.status(401).json({ ok: false, error: "Unauthorized" });
  next();
});
```

Listing 10.2: API key guard (health endpoints exempt)

10.3 Endpoints

10.3.1 GET /health

Returns system status, model configuration, and knowledge base statistics.

```
{
  "ok": true,
  "service": "campuspaddy-retrieval-api",
  "ollama_url": "http://127.0.0.1:11434",
  "ollama_model": "llama3.1:8b",
  "optimized_search": {
    "has_chunk_search_vector": true,
    "has_source_search_vector": true
  },
  "total_chunks": 25345,
  "processed_sources": 11753,
  "active_cached_answers": 2
}
```

Listing 10.3: GET /health response

10.3.2 POST /ask

Main chatbot endpoint.

Request:

```
{
  "question": "How do I contact accommodation?",
  "limit": 3,
  "useCache": true,
  "universityDomain": "tees.ac.uk"
}
```

Response:

```
{
  "ok": true,
  "cached": false,
  "answer": "To contact accommodation at Teesside University,
   you can call 01642 342255 or email accommodation@tees.ac.uk
   ...",
  "sources": [
    { "number": 1, "title": "Contact us | Accommodation |
   Teesside University",
      "url": "https://www.tees.ac.uk/sections/accommodation/
   contact.cfm",
      "source_type": "html" },
    { "number": 2, "title": "Frequently asked questions |
   Accommodation",
```

```
      "url": "https://www.tees.ac.uk/sections/accommodation/faqs.cfm",
      "source_type": "html" }
  ],
  "duration_ms": 5280,
  "retrieval_duration_ms": 950
}
```

10.3.3 POST /retrieve

Returns raw chunks without LLM inference — useful for debugging retrieval quality.

10.3.4 GET /cache/stats

Returns cache metrics: total cached answers, total hits, and average hits per question.

10.3.5 POST /cache/clear

Deactivates all cache entries (sets `is_active = false`) for fresh generation.

10.4 Error Handling

Table 10.1: HTTP status codes

Code	Scenario	Response body
200	Success	{"ok":true,...}
400	Missing question parameter	{"ok":false,"error":"..."}
401	Invalid or missing API key	{"ok":false,"error":"Unauthorized"}
503	Database or Ollama unreachable	{"ok":false,"error":"..."}

Chapter 11

Production Deployment

11.1 AI Server Setup

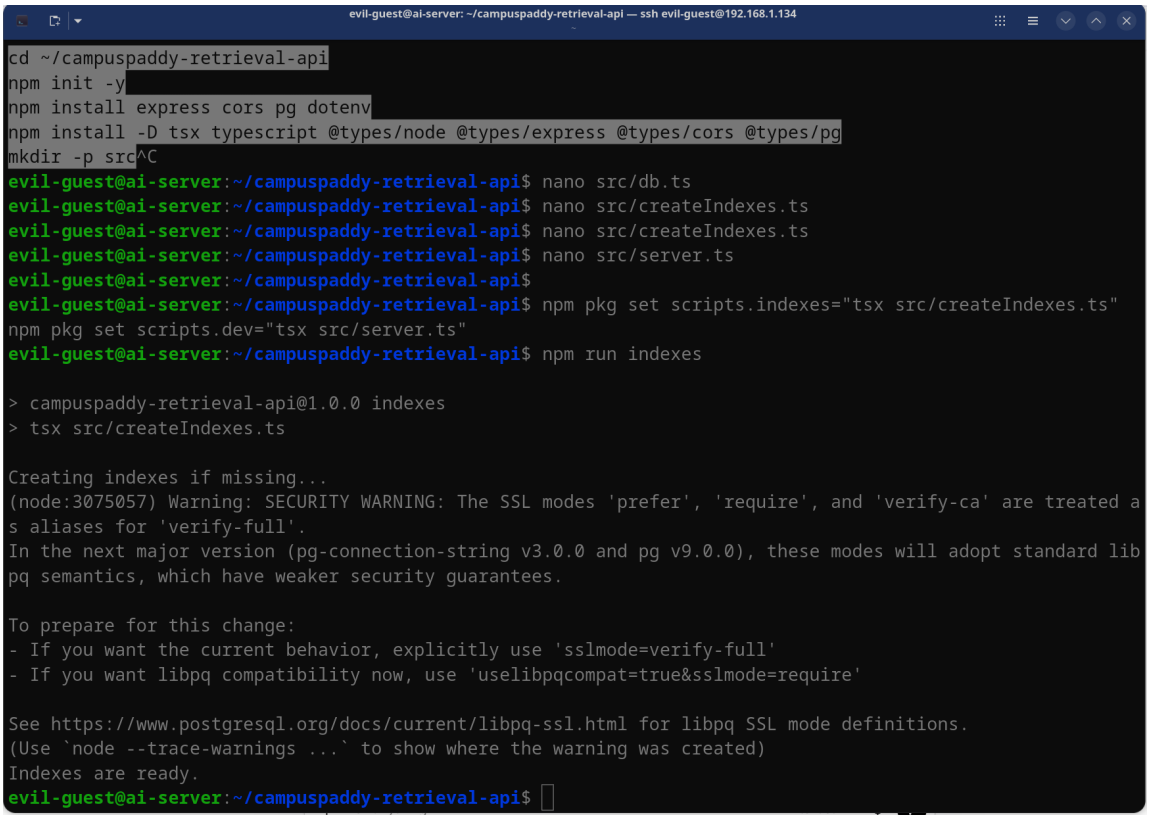


Figure 11.1: AI server terminal: `npm init`, installing Express/CORS/pg/dotenv, creating TypeScript source files, and running `npm run indexes` to create the database indexes in Neon — "Indexes are ready"

11.2 systemd Service

A systemd unit file makes the API start automatically on server boot and restart automatically after any crash:

```
[Unit]
Description=CampusPaddy AI Retrieval API
After=network-online.target ollama.service cloudflared.service
Wants=network-online.target

[Service]
Type=simple
User=evil-guest
WorkingDirectory=/home/evil-guest/campuspaddy-retrieval-api
ExecStart=/usr/bin/npm run dev
Environment="NODE_ENV=production"
Restart=always
RestartSec=5s
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Listing 11.1: /etc/systemd/system/campuspaddy-ai-api.service

```
sudo systemctl daemon-reload
sudo systemctl enable campuspaddy-ai-api
sudo systemctl start  campuspaddy-ai-api
sudo journalctl -u campuspaddy-ai-api -f   # follow live logs
```

Listing 11.2: Enabling and starting the service

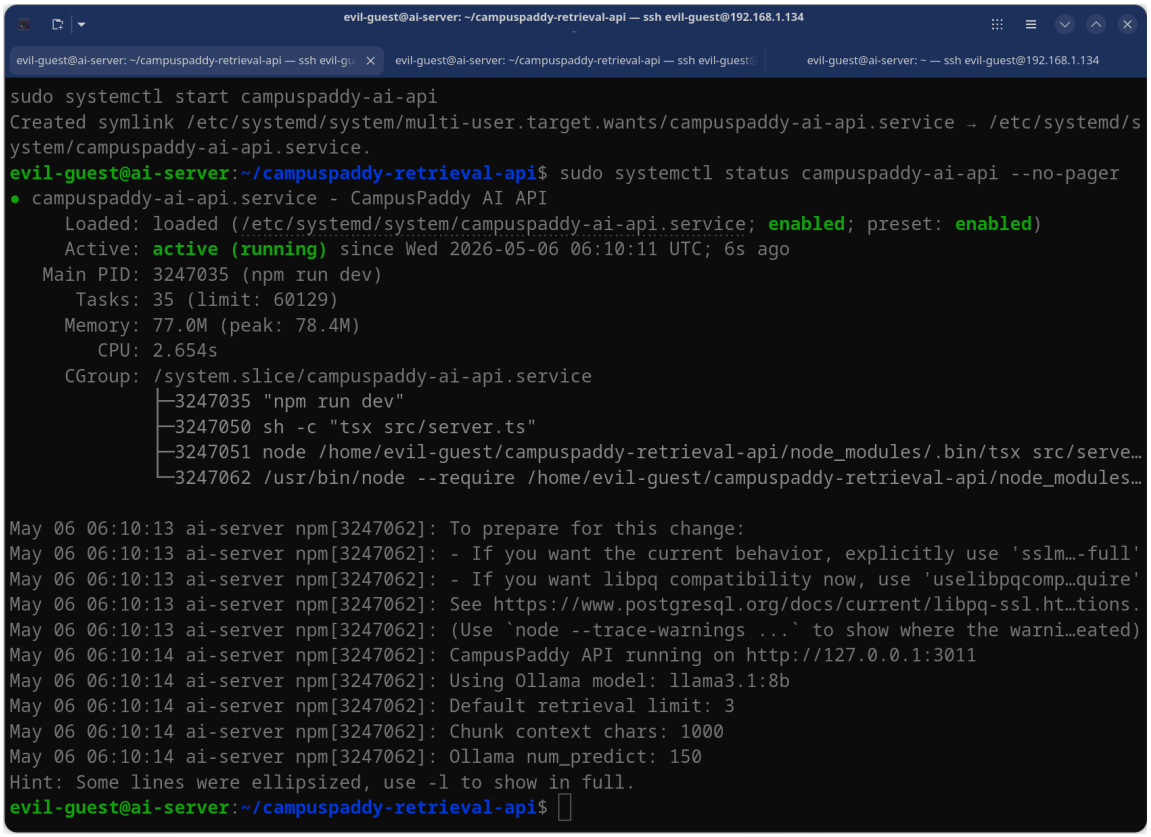


Figure 11.2: Terminal: `sudo systemctl status campuspaddy-ai-api --no-pager` — service shown as **active (running)** since Wed 2026-05-06 06:10:11 UTC, with the Node.js process tree visible (npm → tsx → node)

11.3 Cloudflare Tunnel

The API binds only to `localhost` to prevent direct internet access. Cloudflare Tunnel creates an encrypted outbound connection from the server to Cloudflare's edge, which maps the public hostname to the local port — no firewall rules needed, no open inbound ports.

```
tunnel: 01077474-1853-460b-9744-7f5136679ea2
credentials-file: /root/.cloudflared/01077474-...json

ingress:
  - hostname: campuspaddy-ai.edjenuwahome.uk
    service:  http://localhost:3011
  - service:  http_status:404
```

Listing 11.3: /etc/cloudflared/config.yml

```
# Route the public hostname to the tunnel
cloudflared tunnel route dns 01077474-... campuspaddy-ai.
    edjenuwahome.uk

# Enable tunnel auto-start
```

```
sudo systemctl enable cloudflared
sudo systemctl start  cloudflared
```

Chapter 12

Performance Optimisation

12.1 Before and After

The initial implementation used `ILIKE` pattern matching for retrieval, which requires a full table scan over 25,000 rows:

Table 12.1: Performance before and after optimisation

Component	Before	After	Improvement
Retrieval	7–12 s	≈0.95 s	88% faster
Inference	4–5 s	≈4.3 s	marginal
Total (cold)	13–16 s	≈5.3 s	68% faster
Cached	0.2 s	0.2 s	maintained

12.2 Optimisation Techniques

1. GIN-Indexed `tsvector` Columns

Pre-computing the search vector and storing it in a `GENERATED ALWAYS AS...STORED` column eliminates per-query `to_tsvector()` calls and allows the GIN index to be used:

```
-- Before: computed at query time (no index usable)
WHERE to_tsvector('english', chunk_text) @@ query

-- After: GENERATED column + GIN index
ALTER TABLE knowledge_chunks
  ADD COLUMN chunk_search_vector tsvector
  GENERATED ALWAYS AS (to_tsvector('english', chunk_text))
   STORED;

CREATE INDEX chunk_fts_gin ON knowledge_chunks USING GIN(
   chunk_search_vector);
```

2. Candidate Oversample then Diversify

Fetching 18 candidates (6× the final limit) and then applying diversification gives better result quality than fetching exactly 3, because it allows the diversification algorithm to choose the best non-redundant set.

3. Connection Pooling

```
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: true },
  max: 10,      // max simultaneous connections to Neon
  idleTimeoutMillis:   30000,
  connectionTimeoutMillis: 10000
});
```

4. Ollama `keep_alive`

Setting `keep_alive: "24h"` prevents the model from being evicted from VRAM between requests. Without this, the first request after a quiet period incurs a 2–3 s model reload.

12.3 Latency Breakdown

Table 12.2: Profiling of a typical uncached request

Component	Duration (ms)	Share
Database retrieval	950	17.9%
Chunk diversification	10	0.2%
Context string build	20	0.4%
Ollama network overhead	200	3.8%
Ollama token generation	4100	77.4%
Response serialisation	20	0.4%
Total	**5300**	**100%**

LLM token generation dominates. Further improvement would require streaming responses (SSE), a smaller model, or a faster GPU.

Chapter 13

Results and Evidence

13.1 API Health Check

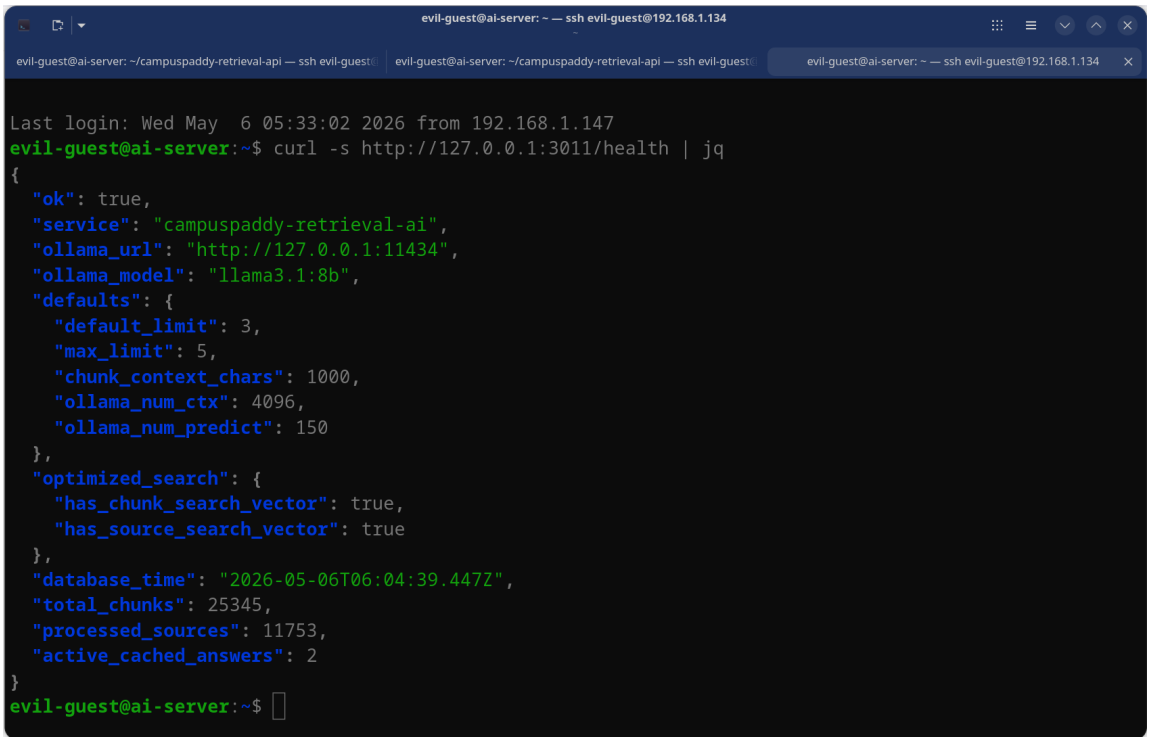


Figure 13.1: `curl -s http://127.0.0.1:3011/health | jq`: API health response confirming 25,345 chunks, 11,753 processed sources, 2 active cached answers, and both `chunk_search_vector` and `source_search_vector` optimisations active

13.2 First Live API Response

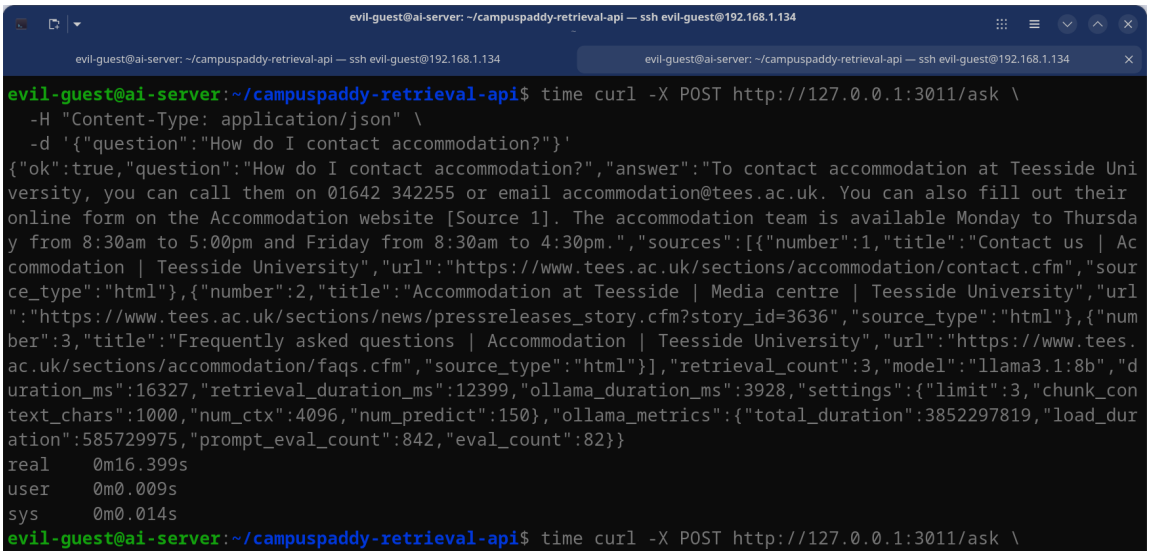
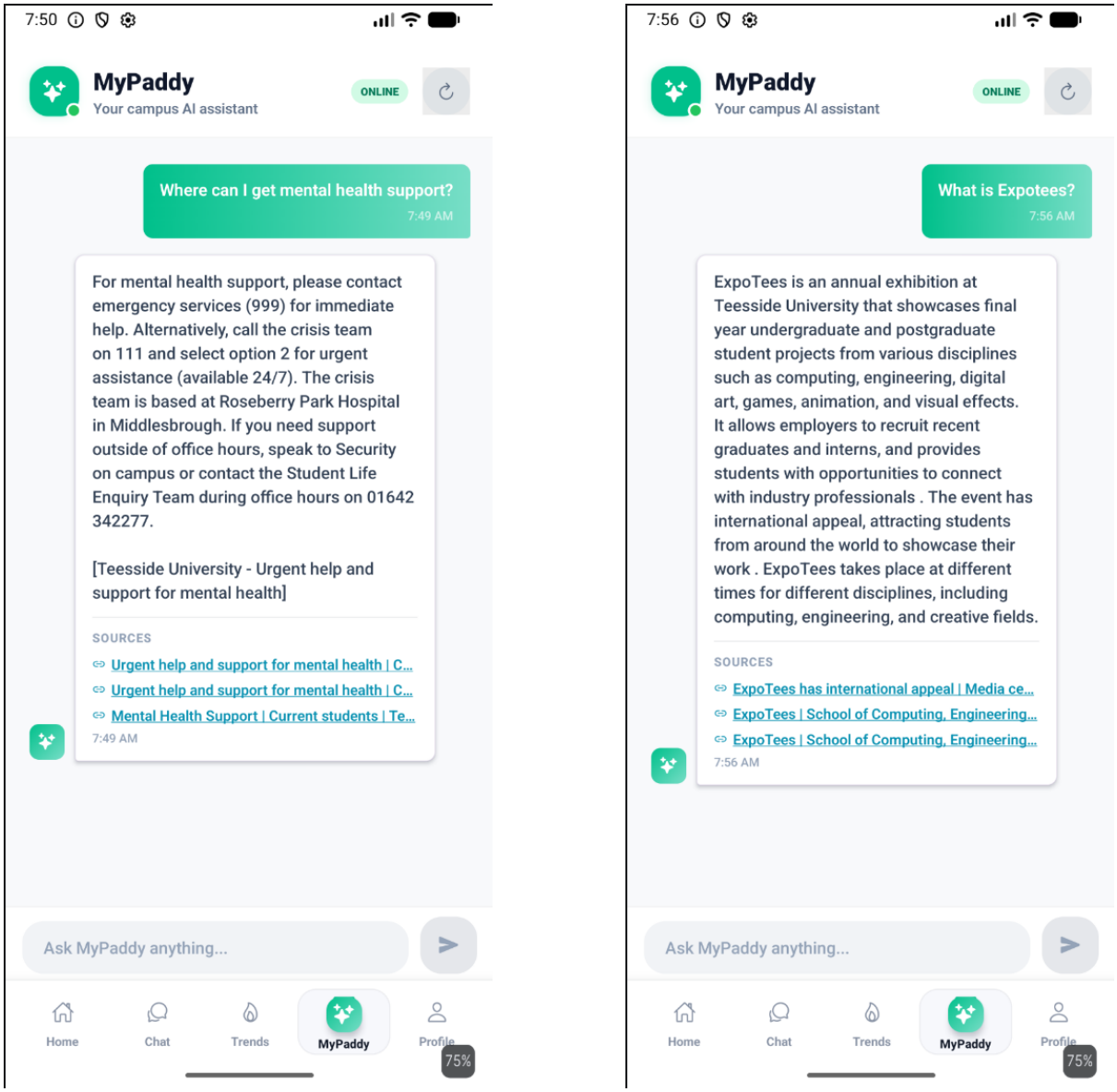


Figure 13.2: First `POST /ask` test via curl: question *"How do I contact accommodation?"* returned with a full answer, three source URLs, retrieval time 16,327 ms and Ollama time 3,928 ms — *before* the tsvector index optimisation was applied (pre-optimisation baseline showing slow retrieval)

13.3 Live Chatbot in the Mobile App

The API was integrated into the MyPaddy mobile application (a companion student app). The following screenshots demonstrate real answers delivered to a mobile client:



(a) Mental health support question (b) ExpoTees event question

Figure 13.3: MyPaddy mobile app connected to the CampusPaddy API: (a) answer to *"Where can I get mental health support?"* citing the crisis team on 999 and Roseberry Park Hospital; (b) answer to *"What is ExpoTees?"* with three university source links

13.4 Summary of Evidence

Table 13.1: Development screenshot timeline

Date / Time (UTC)	Phase	Shots
May 5, 11:55–18:12	Schema research and initial DB review	3
May 6, 05:34–05:43	Adminer DB inspection (schema, tables)	5
May 6, 05:46–06:04	Data browsing (sources, chunks, counts)	5
May 6, 06:32–06:45	AI server setup and index creation	6
May 6, 06:51–07:10	API testing, health check, systemd	5
May 6, 07:50–07:56	Live mobile app integration	4
Total		**31**

Chapter 14

Security Considerations

14.1 API Key Management

The `x-api-key` header protects write endpoints and cache management. Keys are generated with:

```
openssl rand -hex 32
```

Keys are stored only in `.env` files, which are excluded from version control via `.gitignore`.

14.2 No Open Inbound Ports

The server runs *no* open inbound ports. The Cloudflare Tunnel creates an outbound connection only. This means:

- No SSH brute-force on the API port
- No need for `ufw`/iptables rules for the API
- TLS is handled by Cloudflare (valid certificate, HSTS)

14.3 Secrets Never Committed

```
.env
.env.local
.env.*.local
.cloudflared/
cloudflare-credentials.json
credentials.json
secrets.json
```

Listing 14.1: .gitignore entries protecting credentials

All repository secrets are provided through `.env.example` template files with placeholder values only.

14.4 Data Privacy

The crawler does not access authenticated areas (login, student portals, Blackboard, e-vision). All scraped content is publicly accessible without credentials — the same content any visitor to `tees.ac.uk` would see.

Chapter 15

Conclusion and Future Work

15.1 What Was Achieved

Starting from an idea conceived in second year with no clear implementation path, I built a complete, production-deployed RAG chatbot for Teesside University students. The final system:

- Ethically scraped and indexed **11,753 pages** from `tees.ac.uk`
- Produced a knowledge base of **25,345 text chunks**
- Achieves a **97.4% processing success rate**
- Retrieves relevant context in **≈0.95 s** (down from 7–12 s)
- Answers uncached questions in **≈5.3 s** end-to-end
- Serves **cached answers in ≈0.2 s** (26.5× faster)
- Runs **24/7 on a public HTTPS endpoint** with zero cloud LLM cost

The biggest engineering lessons were:

1. **Ethical scraping is not optional.** Rate limiting and robots.txt compliance protect both the target server and the legitimacy of the project.
2. **Full-text search is highly capable.** PostgreSQL `tsvector` with GIN indexes delivered 88% retrieval speedup without any external dependency.
3. **Caching is transformative.** A 14-day persistent cache reduced the majority of user-facing latency to a fraction of the cold-path cost.
4. **Separation of concerns is essential.** Running the scraper separately from the API allowed each to be iterated on independently.

15.2 Future Work

Short term Streaming responses via Server-Sent Events (SSE) to show the answer as it is generated, eliminating the perceived 5-second wait.

Medium term `pgvector` semantic search alongside full-text search, giving the retriever both keyword precision and semantic recall.

Medium term A web UI making CampusPaddy accessible without requiring an API client.

Long term Scheduled re-crawling with change detection, so the knowledge base stays current as the university website updates.

Long term Extending the multi-university schema to serve other institutions by adding new `universities` rows and running the same pipeline against their domains.

Appendix A

Installation Guide

A.1 Scraper Server

```
# 1. System dependencies
sudo apt update && sudo apt install -y postgresql nodejs npm
   git

# 2. Database setup
sudo -u postgres psql <<'SQL'
CREATE USER campuspaddy WITH PASSWORD 'campuspaddy';
CREATE DATABASE campuspaddy_local OWNER campuspaddy;
SQL

# 3. Project setup
cd campuspaddy-ai-importer
cp .env.example .env          # edit DATABASE_URL
npm install
psql postgresql://campuspaddy:campuspaddy@localhost/
   campuspaddy_local \
  -f db/schema.sql

# 4. Run pipeline
npm run crawl:tees            # 6-8 hours
npm run process:all           # 2-4 hours
npm run report                # view statistics
```

A.2 AI Server

```
# 1. Install Ollama and pull model
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3.1:8b

# 2. Project setup
cd campuspaddy-retrieval-api
```

```
cp .env.example .env          # set NEON_DATABASE_URL, API_KEY
npm install
npm run indexes               # create DB indexes
npm run optimize:search       # add tsvector columns + GIN
   indexes

# 3. Test locally
npm run dev &
curl -X POST http://127.0.0.1:3011/ask \
  -H 'Content-Type: application/json' \
  -d '{"question":"What is ExpoTees?"}'
```

Appendix B

API Reference

Base URL: https://campuspaddy-ai.edjenuwahome.uk

Table B.1: Endpoint reference

Method	Path	Auth	Description
GET	/health	None	System health and stats
GET	/ollama/health	None	Ollama model status
GET	/cache/stats	API key	Cache hit metrics
POST	/cache/clear	API key	Deactivate all cache entries
POST	/retrieve	API key	Retrieve chunks (no LLM)
POST	/ask	API key	Full RAG question answering

Appendix C

Repository

GitHub Repository	https://github.com/bahdleen/Teesside-Chatbot
Public API	https://campuspaddy-ai.edjenuwahome.uk
Licence	MIT
Author	Anthony Efemena Edjenuwa
University	Teesside University, Middlesbrough

```
git clone https://github.com/bahdleen/Teesside-Chatbot.git
cd Teesside-Chatbot
```

Listing C.1: Clone and explore